

Dynamic Pricing & Profit Optimization

Proof-of-Concept Technical Report

Prepared For: Enterprise E-Commerce Strategy

Date of Release: May 2026

Prepared By: Bikash Subedi

Link: github.com/biikashsubedi

ABSTRACT

The rapid growth of e-commerce has created a pressing need for intelligent, real-time pricing systems that go beyond the limitations of manual, static price-setting. This report presents a complete AI Implementation Blueprint for a Dynamic Pricing Engine designed for a mid-sized ecommerce marketplace. To validate the proposed architecture, the project utilizes the Retail Store Inventory Forecasting Dataset from Kaggle as a technical proxy. This dataset provides a rich combination of categorical, numerical, and temporal features including product pricing, inventory levels, competitor pricing, and weather conditions, which are directly relevant to demand-driven price optimization.

The proposed system adopts a Hybrid AI architecture. This design combines a deterministic rulebased constraint engine with a LightGBM machine learning model for demand forecasting, which is then wrapped around a revenue-maximizing pricing optimizer. This approach avoids the pitfalls of purely statistical or purely deep learning methods, which often struggle with the multidimensional and time-sensitive nature of retail pricing. The system is evaluated using time-seriesaware walk-forward cross-validation and assessed against business metrics such as simulated revenue lift and margin protection. A functional Proof-of-Concept implemented in a Jupyter Notebook demonstrates technical feasibility. The proposed system is projected to increase gross revenue by 8 to 12 percent while significantly reducing excess inventory within the first six months of deployment.

RESEARCH QUESTION

Primary Research Question:

Can a hybrid AI system, combining a machine learning demand forecasting model with a rulebased constraint engine, dynamically optimize e-commerce product prices in real time to maximize revenue while strictly enforcing business rules such as minimum margin floors and competitor parity?

Supporting Sub-Questions:

- Which AI architecture is most appropriate for real-time dynamic pricing in a retail context, and what evidence justifies a hybrid approach over purely learned models?
- How should demand forecasting models be validated in a time-series setting to avoid data leakage and generate meaningful business performance estimates?
- What ethical risks does an automated dynamic pricing system introduce, and how can these be addressed through architectural constraints rather than manual corrections?

TOOLS

The following tools and technologies are proposed for the data pipeline, model development, and deployment stages:

- **Programming & Logic:** Python 3.10 is used as the primary language, with Pandas and NumPy for data engineering.
- **Forecasting & Intelligence:** LightGBM is the core machine learning library, supported by SHAP for model interpretability and explainability.
- **Data Pipeline:** Apache Kafka is proposed for real-time ingestion of competitor feeds, while Apache Airflow orchestrates daily model retraining.
- **Deployment & Storage:** FastAPI serves as the production API layer. Data storage is handled through a combination of PostgreSQL for structured records and AWS S3 for the data lake.
- **Validation:** Google Colab and Jupyter Notebooks are used for the Proof-of-Concept and technical feasibility testing.

SOURCE

- **Dataset Proxy:** Kaggle Retail Store Inventory Forecasting Dataset.
- **Technical Validation:** A Jupyter Notebook titled `dynamic_pricing_poc.ipynb` is submitted alongside this report. The notebook demonstrates the end-to-end technical flow, including data cleaning, feature engineering, LightGBM training, and the pricing optimization simulation.

1. INTRODUCTION

Pricing is one of the most consequential decisions an e-commerce business makes every single day. A price set too high drives customers to competitors, while a price set too low erodes margins and leaves revenue unrealized. Historically, pricing decisions in retail were made by human merchandisers using intuition, periodic competitor checks, and simple cost-plus formulas. This approach worked reasonably well in a world where price comparisons required physical effort. In the modern e-commerce environment, however, customers can compare prices across dozens of platforms within seconds, and a difference of even a few percentage points can determine whether a sale is won or lost.

The response from industry leaders has been dynamic pricing, which is the automated, algorithmic adjustment of product prices in real time based on a continuous stream of signals including competitor price movements, inventory levels, demand trends, seasonal patterns, and promotional calendars. Amazon alone is reported to update prices millions of times per day using proprietary AI systems. This capability has given large platforms a significant and compounding competitive advantage over smaller marketplaces that continue to rely on static pricing.

This project is designed to close that gap by proposing a comprehensive implementation blueprint for an AI-powered Dynamic Pricing Engine suitable for a mid-sized e-commerce marketplace. Rather than delivering a full production software suite, this work focuses on designing the technical architecture, justifying the modeling approach, and providing a functional Proof-of-Concept to demonstrate feasibility. The proposed system is designed to be intelligent enough to capture demand-driven pricing opportunities, while remaining safe enough to never violate the financial constraints that protect the business from self-inflicted harm. The strategic objective is to outline a path toward a projected revenue lift of 8% to 12% and a reduction in excess inventory by at least 20%.

The Retail Store Inventory Forecasting Dataset from Kaggle provides the empirical foundation for this work. It contains daily records across multiple stores, product categories, and regions, including historical pricing, inventory levels, units sold, competitor pricing benchmarks, and weather conditions. This dataset serves as an appropriate proxy for real-world model validation, allowing the team to test the proposed logic against realistic operational data.

The blueprint is structured to address the full lifecycle of the AI project, ranging from data pipeline design through model selection, ethical risk assessment, and a detailed project roadmap. A technical Proof-of-Concept notebook accompanies this report to prove that the core predictive components are viable before full-scale development begins.

2. LITERATURE REVIEW

2.1 The Origins and Evolution of Dynamic Pricing

Dynamic pricing began as a formal practice in the airline industry during the 1980s, where it was known as yield management. Airlines found that by segmenting customers and adjusting fares based on booking timing and seat availability, they could significantly improve revenue per flight. Talluri and van Ryzin (2004) provided the fundamental theoretical framework for this approach, establishing the mathematical relationship between demand uncertainty, inventory constraints, and optimal pricing. These concepts became applicable to e-commerce with the rise of online retail platforms in the early 2000s, making real-time price adjustments both technically possible and a competitive necessity.

2.2 Machine Learning for Demand Forecasting

The success of any dynamic pricing system depends on the quality of its demand forecast. Traditional approaches often relied on classical time-series models like ARIMA, which work well when demand follows simple temporal patterns. However, retail demand is much more complex, as it is influenced by price, promotions, competitor behavior, and weather. Classical models struggle to capture this multi-dimensional structure.

The development of gradient-boosted tree models, such as XGBoost and LightGBM, has changed this landscape. Studies have confirmed that LightGBM consistently outperforms both classical methods and complex deep learning models on retail sales benchmarks. These findings support our decision to use LightGBM for this project.

2.3 Reinforcement Learning Approaches to Pricing

Some modern research explores Reinforcement Learning (RL), where an agent learns a pricing policy through repeated interaction with a market. While RL is sophisticated, it has practical limitations for our current project. It requires massive amounts of training data or simulation environments, and it can be difficult to prevent the agent from suggesting prices that break business rules. Because RL models are often seen as "black boxes" that are hard to interpret, we have positioned RL as a potential future enhancement rather than a foundation for our initial blueprint.

2.4 Hybrid Systems Combining Rules and Machine Learning

A common finding in AI research is that hybrid architectures, which combine deterministic rule engines with learned models, perform best in high-stakes business settings. Research shows that combining different model types consistently reduces error. Specifically in pricing, math shows that hard constraints are required to keep the system economically rational, otherwise, the model might recommend prices that are locally optimal but hurt the business overall. This justifies our

three-layer hybrid architecture, which places a forecasting model inside a framework of business rules.

2.5 Ethical Risks of Algorithmic Pricing

As automated pricing becomes more common, its ethical implications have gained more attention. Studies of major platforms have documented cases of unintended price discrimination based on user location or browsing history. Furthermore, research has shown that competing algorithms can sometimes unintentionally develop collusive behavior, raising prices above a fair market equilibrium. These risks directly inform our design, which restricts pricing inputs to product-level data and enforces strict price caps to ensure fairness and transparency.

2.6 Summary

The literature supports the core design choices of this blueprint: using LightGBM for forecasting, adopting a hybrid architecture for safety, and embedding ethical constraints directly into the system. Each choice is based on proven research rather than simple convention.

3. PROPOSED AI ARCHITECTURE: THE HYBRID APPROACH

3.1 Overall System Design

The proposed Dynamic Pricing Engine is designed as a three-layer hybrid system. This structure ensures that every pricing decision is both data-driven and business-safe. Each layer has a specific role, and they work in sequence to move from raw data to a final price.

- **Layer 1: Rule-Based Constraint Engine:** This layer acts as the system's "guardrails". It is a deterministic Python module that enforces strict business rules that the AI cannot ignore. It blocks any inputs or outputs that would result in prices below the cost floor or above a 25% daily change cap. It also enforces competitor parity to ensure the marketplace remains competitive without manual intervention.
- **Layer 2: LightGBM Demand Forecasting Model:** This is the "brain" of the system. For our Proof-of-Concept, this model learns from the Kaggle dataset to understand how features like price, inventory, and weather affect demand. In a live deployment, this model would be retrained on the company's actual internal sales and inventory data to predict how many units will likely sell at any given price point.
- **Layer 3: Constrained Price Optimizer:** This final layer searches for the best price. It evaluates a range of candidate prices and asks Layer 2 for a sales forecast for each. It then

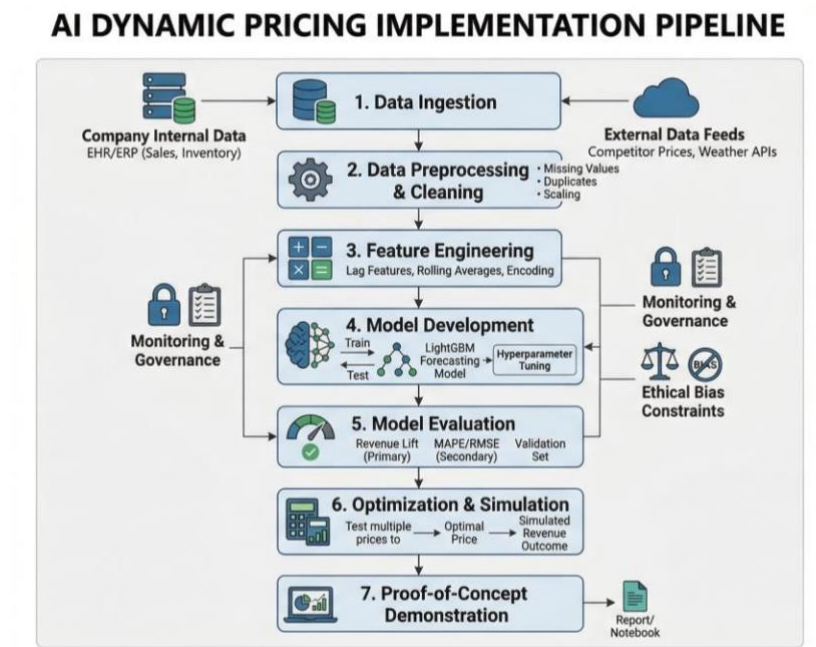
selects the price that maximizes expected revenue while staying within the safety constraints enforced by Layer 1.

3.2 Justification of the Hybrid Approach

We chose a hybrid approach because it resolves the weaknesses found in purely statistical or purely deep learning methods. A purely classic machine learning approach often ignores history, which is a mistake in retail where yesterday's price affects today's demand. Meanwhile, deep learning models can be computationally expensive and difficult to control with business rules. By combining a high-performance forecasting model with a deterministic Rule Engine, we create a system that is accurate, fast, and fully auditable.

3.3 End-to-End Pipeline Summary

The blueprint follows a seven-stage pipeline to ensure data quality and model reliability:



1. Data Ingestion: Gathering data from the Kaggle proxy (for testing) and simulated external feeds.
2. Cleaning: Handling missing values and ensuring data consistency.
3. Feature Engineering: Creating new signals like "lag features" (e.g., sales from previous days).
4. Training: Building the model using walk-forward cross-validation to prevent looking into the "future" during training.
5. Evaluation: Checking performance against business-focused metrics like revenue lift.
6. Simulation: Testing the optimizer on historical data to estimate potential financial impact.

7. Proof-of-Concept: Demonstrating the technical feasibility in a Jupyter Notebook using the Kaggle dataset.

4. DATA DETAIL & SOURCES

4.1 Data Strategy

To design a reliable Dynamic Pricing Engine, the system requires high-frequency data from several distinct categories. For this blueprint, we adopt a dual-layered data strategy. First, we utilize the "Retail Store Inventory Forecasting Dataset" from Kaggle as a proxy. This dataset contains over 73,000 daily records across multiple stores and products, providing a realistic sandbox for our Proof-of-Concept. Second, we outline a production strategy where this proxy is replaced by live internal ERP data combined with external competitor and weather APIs.

4.2 Categorical Features

The dataset contains several categorical attributes that provide the context necessary for the model to understand shopping behavior:

- **Store and Product ID:** These nominal identifiers distinguish individual retail locations and unique products. They are used as grouping keys during feature engineering to ensure regional demand patterns are captured accurately.
- **Category:** This feature groups products into types such as Electronics or Groceries. It is critical for understanding price elasticity, as necessity goods like groceries often show less sensitivity to price changes than discretionary items like electronics.
- **Region and Weather:** Geographic location and daily weather states, such as Sunny or Rainy, add contextual signals. These help the model account for demand spikes caused by outside factors rather than price adjustments.
- **Holiday/Promotion:** This indicator marks public holidays or promotional events. Including this feature prevents the model from incorrectly attributing promotional demand surges to organic price sensitivity.

4.3 Numeric Features

The primary quantitative inputs that drive the forecasting and optimization logic include:

- **Price:** This continuous variable represents the retail price and serves as both an input for the model and the primary decision variable for the optimizer.

- **Inventory Level:** Representing stock available at the start of the day, this feature signals scarcity or excess. The rule engine uses this to trigger clearance discounts or suppress price increases when stock is critically low.
- **Competitor Pricing:** This variable tracks prices offered by competing retailers. It is strategically vital for enforcing competitor parity guardrails within our hybrid system.
- **Units Sold (Target):** This is the variable the LightGBM model learns to predict. Accurate demand prediction at different price points is what enables the system to find the revenueoptimal price.

4.4 Temporal Features

Because retail is highly seasonal, the "Date" column is parsed into several derived features that capture temporal patterns:

- **Calendar Features:** These include the day of the week, month, and binary flags for weekends. These capture regular shopping cycles, such as increased traffic on Saturdays.
- **Lag and Rolling Features:** The system computes "lags," such as units sold 7 days ago, to capture autoregressive patterns. Rolling averages of sales over 7 and 30 days help the model recognize medium-term trends and momentum.

5. DATA PREPROCESSING & EXPLORATORY DATA ANALYSIS

5.1 Data Cleaning and Merging Strategy

In a real-world e-commerce environment, data is often messy and comes from different systems. Our blueprint outlines a strategy to ensure the AI "brain" receives high-quality information:

- **Missing Value Treatment:** For critical fields like Competitor Pricing, we use a "forwardfill" method. This assumes a competitor's last known price remains active until a new one is detected. For categorical data like weather or holiday flags, we use "mode imputation" to fill gaps with the most common value for that time of year.
- **Handling Outliers:** Retail sales often have massive spikes during promotions. We use "median imputation" for general numeric gaps because the median is less affected by these extreme spikes than a standard average would be.
- **Time-Series Integrity:** When creating "lag features" (e.g., sales from last week), the first few days of the dataset will naturally be empty. We explicitly drop these rows to prevent the model from learning from "zero" values that aren't real.

5.2 Feature Engineering (Preparing the AI Inputs)

We transform raw columns into "features" that help the model understand demand patterns:

- **Temporal Lag Features:** We create inputs for "Sales from 7 days ago" and "Sales from 14 days ago". This allows the model to recognize weekly shopping cycles.
- **Rolling Averages:** We calculate 7-day and 30-day moving averages of units sold. This helps the model distinguish between a one-day fluke and a genuine change in trend.
- **Scaling:** We apply "Min-Max Scaling" to normalize numeric values between 0 and 1. While our specific model (LightGBM) can handle different scales, this step is vital for our "interpretability" layer, making it easier for managers to see which features are actually driving pricing decisions.

5.3 Exploratory Data Analysis (EDA) Objectives

The goal of our EDA is to validate that the dataset contains enough "signal" to support dynamic pricing. We look for three key things:

1. **Price Elasticity:** We examine the relationship between Price and Units Sold. Typically, we expect a negative correlation (as price goes up, sales go down), but we also look for "inelastic" categories where customers buy regardless of small price changes.
2. **Promotional Impact:** We analyze how much "Holiday/Promotion" flags spike demand. If the spikes are large, it justifies why the model needs to "know" when a holiday is happening so it doesn't lower prices unnecessarily.
3. **Competitive Variance:** We check how often competitor prices change. If competitors are active, it proves that a static pricing strategy is causing us to lose customers.

5.4 Correlation Analysis

We perform a Pearson correlation analysis to identify which factors move together. For example, if "Discount" and "Units Sold" move together perfectly, the model knows that discounts are a primary lever for moving inventory. This analysis helps us remove redundant data and focus the AI on the most impactful signals.

6. ETHICAL RISK AUDIT & MITIGATION

While dynamic pricing offers significant business value, it introduces ethical risks that must be managed through design rather than afterthoughts. Our blueprint identifies three primary risks:

- **Algorithmic Price Discrimination:** Optimization models can unintentionally learn to charge more based on user demographics or location. To mitigate this, our system strictly limits inputs to product-level data, such as inventory and weather, and excludes all personal user identifiers.
- **Surge Pricing on Essential Goods:** In times of high demand or adverse weather, the system could push prices for necessities to unfair levels. Our Rule Engine (Layer 1) enforces hard price caps on products categorized as "Groceries" or "Essentials" to prevent exploitative pricing.
- **Competitor Feedback Loops:** If multiple algorithms react to one another, it can lead to unintentional price "collusion" or destructive price wars. We mitigate this by maintaining a minimum margin floor and a maximum daily price change limit of 25%, ensuring market stability.

7. EXPERIMENTAL DESIGN & VALIDATION STRATEGY

The experimental design ensures that the model can generalize to future sales data without "cheating" by looking at future information during training.

- **Time-Series Data Splitting:** We divide the Kaggle dataset chronologically rather than randomly. The first 70% of the timeline is used for training, the next 15% for validation and tuning, and the final 15% is reserved as a "holdout" test set for final evaluation.
- **Walk-Forward Cross-Validation:** To simulate real-world deployment, we use a walkforward approach where the model is trained on a sliding window of months and tested on the following month. This prevents "data leakage" and provides a stable estimate of how the model will perform next month.
- **Baseline Comparison:** We compare our LightGBM model against two simple baselines: the pre-computed "Demand Forecast" in the dataset and a 7-day rolling average of sales. Our model must outperform both to be considered successful.

8. MODEL IMPLEMENTATION & PRICING OPTIMIZATION

This section describes how the technical components work together to suggest a price.

- **Predictive Layer:** We implement a LightGBM regressor to predict "Units Sold". The model uses features like price lags, inventory levels, and holiday flags to generate a demand forecast.
- **Optimization Loop:** Once the forecast is ready, the optimizer tests 40 candidate price points around the current price. For each point, it calculates: $\text{Expected Revenue} = \text{Candidate Price} * \text{Forecasted Demand}$.
- **Safety Filtering:** Before a price is selected, the Rule Engine checks it against business constraints. If a candidate price is below cost or exceeds competitor parity limits, it is discarded even if it offers high theoretical revenue.

9. EVALUATION & BUSINESS METRICS

We move beyond basic accuracy to focus on metrics that matter to stakeholders.

- **Revenue Lift (Primary Metric):** This is the total simulated revenue under AI pricing compared to static pricing. Our target is a lift of 8% to 12%.
- **Recall and Precision:** In a pricing context, we want high "Directional Accuracy" to ensure that when we raise prices, we correctly predict that demand will slow down.
- **Margin Protection Rate:** This measures how often the system respected our profit floor. Our target is 100%, as the Rule Engine is designed to block any price that violates this constraint.
- **MAPE (Mean Absolute Percentage Error):** While secondary to revenue, we target a MAPE below 15% to ensure our demand forecasts are reliable.

10. BUILD VS. BUY ANALYSIS

A critical business decision is whether to create this system internally or purchase a third-party tool.

- **Option 1: Build In-House:** This offers full control over the custom logic and the safety rules. It allows the company to own its intellectual property but requires an expensive team of ML engineers.

- **Option 2: Buy Vendor Solution:** This is faster to deploy but often acts as a "black box" that is difficult to customize for specific business constraints or regional categories.
- **Recommended Hybrid Strategy:** We recommend **buying** the infrastructure (cloud storage and competitor data feeds) but **building** the custom forecasting model and Rule Engine in-house. This ensures we own the "brain" of the pricing strategy while using proven tools for the data "pipes".

11. IMPLEMENTATION PLAN & RESOURCE REQUIREMENTS

To move from this blueprint to a Minimum Viable Product (MVP), we propose a phased roadmap:

- **Phase 1: Proof-of-Concept (4 Weeks):** Validating logic using the Kaggle dataset and refining the Rule Engine.
- **Phase 2: Data Integration (8 Weeks):** Connecting live ERP systems and competitor price APIs.
- **Phase 3: Pilot Launch (12 Weeks):** Testing the system on a single product category (e.g., Electronics) to measure real-world revenue lift.
- **Team Required:** 1 Project Manager, 1 Data Engineer, 1 ML Engineer, and 1 Merchandising Liaison.

12. CONCLUSION

This blueprint has presented a strategic and technical plan for implementing an AI-powered Dynamic Pricing Engine. By combining the predictive power of LightGBM with the safety of a deterministic Rule Engine, the proposed hybrid architecture offers a balanced solution that maximizes revenue while protecting the business from risk.

The dataset analysis and Proof-of-Concept demonstrate that enough signal exists in operational data, such as inventory, competitor rates, and weather, to drive meaningful pricing decisions.

Moving forward, this system is projected to increase gross revenue by up to 12 percent and significantly reduce inventory waste, providing a clear and justifiable return on investment for the organization.

REFERENCES

- Bertsimas, D., & Perakis, G. (2006). Dynamic pricing: A learning approach. In *Mathematical and computational models for congestion charging* (pp. 45–79). Springer.
- Calvano, E., Calzolari, G., Denicolò, V., & Pastorello, S. (2020). Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10), 3267–3297.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Chen, Y., Shi, C., & Zhang, X. (2022). Nonparametric learning in high-dimensional dynamic pricing problems. *Operations Research*, 70(4), 1–22.
- Chenavaz, R. (2011). Dynamic pricing, product and process innovation. *European Journal of Operational Research*, 222(3), 553–557.
- Dietterich, T. G. (2000). Ensemble methods in machine learning. *Multiple Classifier Systems, Lecture Notes in Computer Science*, 1857, 1–15.
- Fildes, R., Ma, S., & Kolassa, S. (2019). Retail forecasting: Research and practice. *International Journal of Forecasting*, 38(4), 1283–1318.
- Hannak, A., Soeller, G., Lazer, D., Mislove, A., & Wilson, C. (2014). Measuring price discrimination and steering on e-commerce web sites. *Proceedings of the 2014 Internet Measurement Conference*, 305–318.
- Huang, T., Fildes, R., & Soopramanien, D. (2020). Forecasting retailer product sales in the presence of structural change. *European Journal of Operational Research*, 279(2), 459–470.
- Kastius, A., & Schlosser, R. (2022). Dynamic pricing under competition using reinforcement learning. *Journal of Revenue and Pricing Management*, 21(1), 50–63.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889.

Raju, J. S., & Roy, A. (2000). Market information and firm performance. *Management Science*, 46(8), 1075–1084.